



Carrinho Certo

Planeje sua compra do mês

Documentação do Aplicativo Android

Funções do app · Telas · Banco de dados SQLite
Onde estão os arquivos · O que cada arquivo faz · Testes

Linguagem: Java · Android Studio (Empty Views Activity)
Pacote: `com.example.carrinhocerto`

1. Visão geral do app

O **Carrinho Certo** é um aplicativo Android para planejar a compra do mês no mercado respeitando um orçamento. O usuário informa quanto pode gastar, marca os produtos que vai comprar e o app mostra se o total cabe no orçamento, quanto sobra (ou quanto passou) e dá dicas de economia. Os produtos ficam salvos em um banco de dados **SQLite** e podem ser cadastrados, editados e excluídos (CRUD completo).

Tecnologias usadas

Item	Como foi usado
Linguagem	Java
Telas (Activities)	5 telas, todas estendendo <code>AppCompatActivity</code> e registradas no <code>AndroidManifest.xml</code>
Layouts	XML com <code>LinearLayout</code> dentro de <code>ScrollView</code> , sem bibliotecas externas
Navegação	<code>Intent</code> com dados enviados por <code>putExtra</code> (orçamento, total e id do produto)
Banco de dados	SQLite nativo do Android, com a classe <code>SQLiteOpenHelper</code>
Formato de moeda	Padrão brasileiro com <code>NumberFormat</code> e <code>Locale pt-BR</code> (ex.: R\$ 28,90)
Visual	Tons de verde definidos no <code>colors.xml</code> , botões verdes com texto branco, cartões arredondados
Testes	JUnit + Espresso (testes que rodam no emulador)

Fluxo de navegação entre as telas



Dados enviados entre as telas (Intent)

De	Para	Dado enviado	Chave usada
MainActivity	ListaActivity	Orçamento (double)	<code>Formatador.CHAVE_ORCAMENTO</code>
ListaActivity	ResumoActivity	Orçamento e total (double)	<code>CHAVE_ORCAMENTO</code> , <code>CHAVE_TOTAL</code>
ListaActivity	ProdutoActivity	Id do produto (long) – só ao editar	<code>CHAVE_ID_PRODUTO</code>
DicasActivity	MainActivity	Nenhum (flags que limpam as telas)	<code>FLAG_ACTIVITY_CLEAR_TOP</code> / <code>CLEAR_TASK</code>

2. Telas e funções do app



Tela 1 – Início MainActivity

- Mostra o título **Carrinho Certo** e o subtítulo *Planeje sua compra do mês*.
- Campo numérico decimal para digitar o **orçamento do mês** (aceita vírgula ou ponto, ex.: 500,50).
- Botão **Começar**: abre a Lista de Compras enviando o orçamento pela Intent.
- **Validação**: se o campo estiver vazio ou com zero, aparece um **Toast** pedindo o valor e a tela não avança.

Métodos: `onCreate()` liga o layout e o botão; `comecar()` lê, valida e abre a próxima tela.

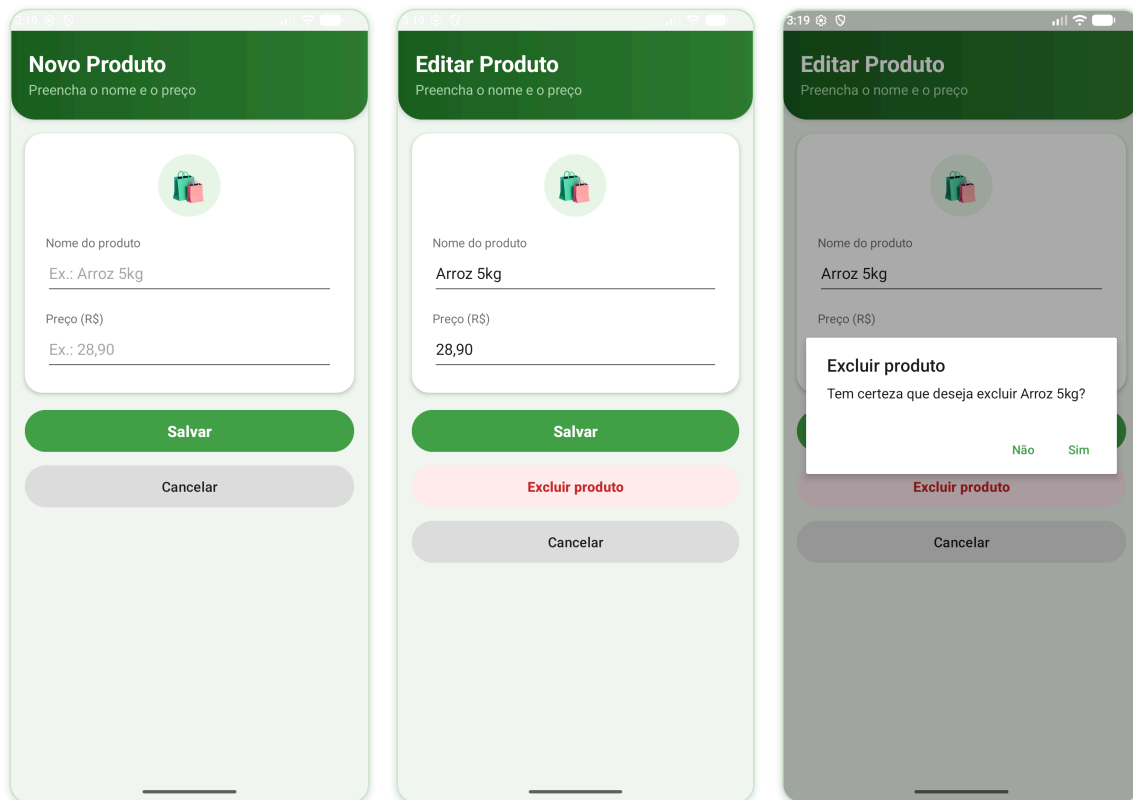


Tela 2 – Lista de Compras ListaActivity

- Carrega os produtos do **banco SQLite** (20 produtos iniciais) e mostra cada um em um cartão com **CheckBox**, nome, preço e botão **Editar**.
- Tocar no cartão marca/desmarca o produto.
- O rodapé mostra o **total marcado** e a quantidade de itens (ex.: *4 de 20 itens*), atualizando **na hora**.
- Botão **+ Novo Produto**: abre a tela de cadastro.
- Botão **Editar**: abre a tela de edição daquele produto.
- A lista é **recarregada no onResume()**, então ao voltar do cadastro/edição as mudanças aparecem na hora, e os produtos marcados continuam marcados.
- Botão **Ver Resumo**: abre o resumo enviando orçamento e total.

Tela de Produto – cadastro, edição e exclusão ProdutoActivity

A mesma tela serve para **cadastrar** (aberta pelo botão + *Novo Produto*, campos vazios) e para **editar/excluir** (aberta pelo botão *Editar*, recebe o id do produto e já vem preenchida).



Novo Produto (cadastrar)

Editar Produto (dados preenchidos)

AlertDialog de confirmação

- **Salvar:** valida (nome obrigatório e preço maior que zero, senão mostra Toast) e grava no banco – `inserir()` se for novo ou `atualizar()` se for edição.
- **Excluir produto:** só aparece na edição; abre um **AlertDialog** “Tem certeza que deseja excluir...?” com **Sim** (exclui) e **Não** (cancela).
- **Cancelar:** volta para a lista sem salvar.

Métodos: `onCreate()` decide se é modo novo ou editar; `salvar()` valida e grava; `confirmarExclusao()` mostra o AlertDialog e exclui; `onDestroy()` fecha o banco.

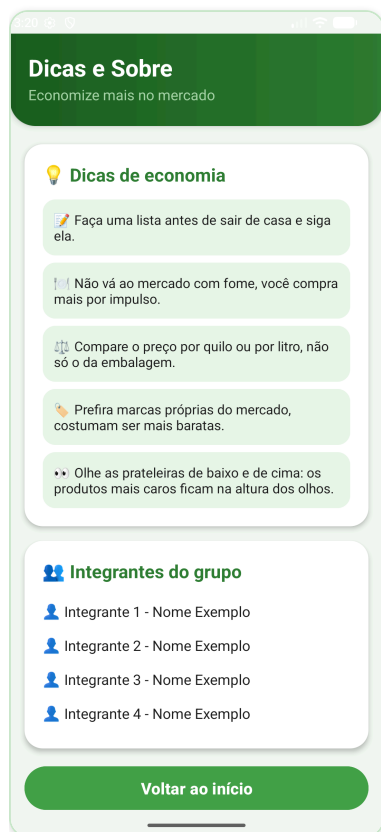
Tela 3 – Resumo da compra ResumoActivity



Dentro do orçamento (verde)

Passou do orçamento (vermelho)

- Mostra o **total da compra**, o **orçamento** e o **saldo restante** (orçamento – total).
- **ProgressBar horizontal** com o percentual do orçamento usado (fica cheia se passar de 100%).
- **Dentro do orçamento:** barra verde e a mensagem “Parabéns, você está dentro do orçamento!”.
- **Passou do orçamento:** barra vermelha e a mensagem “Cuidado, você passou do orçamento em R\$ X”, mostrando quanto passou.
- Botão **Ver Dicas** abre a tela de dicas; botão **Voltar para a lista** fecha o resumo com `finish()` (os produtos continuam marcados).



Tela 4 – Dicas e Sobre DicasActivity

- Mostra **5 dicas de economia** no mercado.
- Seção **Integrantes do grupo** com 4 nomes (para trocar, edite `integrante_1` a `integrante_4` no `strings.xml`).
- Botão **Voltar ao início**: volta para a tela inicial **limpando as telas anteriores** (flags `CLEAR_TOP`, `NEW_TASK` e `CLEAR_TASK`), então o botão voltar do celular não retorna para elas.

Métodos: `onCreate()` liga o layout e o botão Voltar ao início.

3. CRUD – onde cada operação acontece

Operação	Na tela (botão)	No código (Activity)	No banco (DatabaseHelper)
C – Create	+ Novo Produto → Salvar	<code>ProdutoActivity.salvar()</code>	<code>inserir(Produto)</code> – INSERT
R – Read	Abrir a Lista / tocar em Editar	<code>ListaActivity.carregarProdutos()</code>	<code>listarProdutos()</code> e <code>buscarPorId(id)</code> – SELECT
U – Update	Editar → Salvar	<code>ProdutoActivity.salvar()</code>	<code>atualizar(Produto)</code> – UPDATE
D – Delete	Editar → Excluir produto → Sim	<code>ProdutoActivity.confirmarExclusao()</code>	<code>excluir(id)</code> – DELETE

4. Banco de dados SQLite

Tabela produtos

Coluna	Tipo	Descrição
id	INTEGER PRIMARY KEY AUTOINCREMENT	Código do produto, gerado automaticamente
nome	TEXT NOT NULL	Nome do produto (ex.: Arroz 5kg)
preco	REAL NOT NULL	Preço do produto (ex.: 28.90)

Comando que cria a tabela (em `DatabaseHelper.onCreate()`):

```
CREATE TABLE produtos (id INTEGER PRIMARY KEY AUTOINCREMENT, nome TEXT NOT NULL, preco REAL NOT NULL);
```

Produtos iniciais (20)

Versão 1 do banco (10 originais)	Preço	Versão 2 do banco (10 extras)	Preço
Arroz 5kg	R\$ 28,90	Pão de forma 500g	R\$ 8,99
Feijão 1kg	R\$ 8,50	Ovos 12 unidades	R\$ 11,90
Café 500g	R\$ 18,90	Manteiga 200g	R\$ 12,50
Açúcar 5kg	R\$ 22,00	Queijo mussarela 500g	R\$ 24,90
Óleo 900ml	R\$ 7,90	Banana 1kg	R\$ 6,49
Leite 1L	R\$ 5,49	Tomate 1kg	R\$ 7,99
Carne 1kg	R\$ 42,90	Batata 1kg	R\$ 5,99
Frango 1kg	R\$ 15,90	Papel higiênico 12 rolos	R\$ 21,90
Macarrão 500g	R\$ 4,99	Detergente 500ml	R\$ 2,79
Sabão em pó 1kg	R\$ 14,90	Refrigerante 2L	R\$ 9,49

Versões do banco: na primeira instalação, o `onCreate()` cria a tabela com os 20 produtos. Se o app já estava instalado com a versão 1, o `onUpgrade()` só **adiciona** os 10 extras, sem apagar o que o usuário cadastrou.

Onde fica o arquivo do banco

Local	Caminho
Dentro do celular/emulador (banco real, usado pelo app)	/data/data/com.example.carrinhocerto/databases/carrinho_certo.db
Cópia no computador (arquivo do banco)	C:\Users\mikae\AndroidStudioProjects\CarrinhoCerto\banco_de_dados\carrinho_certo.db
Cópia no computador (comandos SQL,	C:\Users\mikae\AndroidStudioProjects\CarrinhoCerto\banco_de_dados\carrinho_certo.sql

abre no Bloco de
Notas)

Importante: o banco real é criado **dentro do aparelho** na primeira vez que a Lista de Compras é aberta. As cópias na pasta `banco_de_dados` são uma “foto” do banco em um momento – não mudam sozinhas. Desinstalar o app apaga o banco do aparelho.

Como ver o banco ao vivo no Android Studio

Database Inspector: rode o app e abra a Lista → *View* → *Tool Windows* → *App Inspection* → aba *Database Inspector* → `carrinho_certo.db` → tabela `produtos`. Dá para rodar SQL, ex.: `SELECT * FROM produtos WHERE preco > 10`.

Device Explorer: *View* → *Tool Windows* → *Device Explorer* → `data/data/com.example.carrinhocerto/databases/`.

5. Onde estão os arquivos do projeto

Pasta do projeto no computador: `C:\Users\mikae\AndroidStudioProjects\CarrinhoCerto`

```
CarrinhoCerto/
├── app/
│   ├── build.gradle.kts                # configuração do módulo (não alterado)
│   └── src/
│       ├── main/
│       │   ├── AndroidManifest.xml    # registro das 5 telas
│       │   ├── java/com/example/carrinhocerto/ # CÓDIGO JAVA
│       │   │   ├── MainActivity.java   # Tela 1 - Início
│       │   │   ├── ListaActivity.java  # Tela 2 - Lista de Compras
│       │   │   ├── ProdutoActivity.java # Cadastro / edição / exclusão
│       │   │   ├── ResumoActivity.java # Tela 3 - Resumo
│       │   │   ├── DicasActivity.java  # Tela 4 - Dicas e Sobre
│       │   │   ├── DatabaseHelper.java  # Banco de dados SQLite
│       │   │   ├── Produto.java         # Classe modelo do produto
│       │   │   └── Formatador.java       # Formato R$ e chaves da Intent
│       │   └── res/                    # RECURSOS (visual)
│       │       ├── layout/             # desenho das telas (XML)
│       │       │   ├── activity_main.xml
│       │       │   ├── activity_lista.xml
│       │       │   ├── item_produto.xml
│       │       │   ├── activity_produto.xml
│       │       │   ├── activity_resumo.xml
│       │       │   └── activity_dicas.xml
│       │       ├── drawable/           # fundos, cantos arredondados, barra
│       │       │   ├── fundo_gradiente.xml ─┬─ fundo_cabecalho.xml
│       │       │   ├── fundo_rodape.xml   ─┬─ fundo_cartao.xml
│       │       │   ├── fundo_circulo.xml  ─┬─ fundo_mensagem.xml
│       │       │   └── barra_progresso.xml
│       │       ├── values/
│       │       │   ├── colors.xml         # cores verdes
│       │       │   ├── strings.xml        # todos os textos
│       │       │   └── themes.xml         # tema do app
│       │       └── values-night/themes.xml # tema no modo escuro
│       ├── androidTest/java/com/example/carrinhocerto/ # TESTES
│       │   ├── ProdutoCrudTest.java        # teste do CRUD no banco
│       │   ├── ProdutoTelaTest.java         # teste dos botões (Espresso)
│       │   └── ExampleInstrumentedTest.java # exemplo padrão
│       └── test/.../ExampleUnitTest.java    # exemplo padrão
├── banco_de_dados/                         # CÓPIA DO BANCO
│   ├── carrinho_certo.db
│   └── carrinho_certo.sql
├── gradle/libs.versions.toml               # versões das bibliotecas (não alterado)
├── build.gradle.kts · settings.gradle.kts · gradle.properties
└── gradlew · gradlew.bat                  # scripts de build do Gradle
```

O projeto também está no GitHub: <https://github.com/mikaellobatodiass-hue/java-backend> (branch `main`).

6. O que cada arquivo faz

Código Java (`app/src/main/java/com/example/carrinhocerto/`)

Arquivo	O que faz	Métodos principais
MainActivity.java	Tela inicial. Lê e valida o orçamento e abre a lista.	<code>onCreate()</code> – liga layout e botão <code>comecar()</code> – valida (Toast se vazio/zero) e abre a <code>ListaActivity</code>
ListaActivity.java	Lista de compras vinda do banco, com CheckBox, botão Editar, Novo Produto e total em tempo real.	<code>onCreate()</code> – liga componentes e botões <code>onResume()</code> – recarrega a lista sempre que a tela aparece <code>carregarProdutos()</code> – busca no banco e monta os cartões <code>atualizarTotal()</code> – soma os marcados e conta os itens <code>onDestroy()</code> – fecha o banco
ProdutoActivity.java	Cadastra, edita e exclui produtos.	<code>onCreate()</code> – modo novo ou editar <code>salvar()</code> – valida e insere/atualiza <code>confirmarExclusao()</code> – AlertDialog e exclusão <code>onDestroy()</code> – fecha o banco
ResumoActivity.java	Calcula saldo e percentual, pinta a barra e a mensagem de verde ou vermelho.	<code>onCreate()</code> – recebe orçamento/total, calcula, mostra e liga os botões Ver Dicas e Voltar
DicasActivity.java	Mostra dicas e integrantes; volta ao início limpando as telas.	<code>onCreate()</code> – botão Voltar ao início com flags <code>CLEAR_TOP/NEW_TASK/CLEAR_TASK</code>
DatabaseHelper.java	Cria e gerencia o banco SQLite <code>carrinho_certo.db</code> (versão 2).	<code>onCreate()</code> – cria a tabela e os 20 produtos <code>onUpgrade()</code> – adiciona os extras sem apagar dados <code>listarProdutos()</code> – SELECT de todos <code>buscarPorId(id)</code> – SELECT de um <code>inserir(produto)</code> – INSERT <code>atualizar(produto)</code> – UPDATE <code>excluir(id)</code> – DELETE <code>criarValores()</code> / <code>lerProduto()</code> – convertem Produto $\rightleftharpoons$ banco
Produto.java	Classe modelo: representa uma linha da tabela produtos.	Atributos <code>id</code> , <code>nome</code> , <code>preco</code> ; 2 construtores; getters e setters
Formatador.java	Classe de apoio usada por todas as telas.	<code>emReais(valor)</code> – formata em R\$ 0,00 Constantes <code>CHAVE_ORCAMENTO</code> , <code>CHAVE_TOTAL</code> , <code>CHAVE_ID_PRODUTO</code>

Layouts (`app/src/main/res/layout/`)

Arquivo	O que desenha
activity_main.xml	Tela inicial: fundo em degradê, ícone do carrinho, campo do orçamento com “R\$” e botão Começar.
activity_lista.xml	Lista: cabeçalho, botão + Novo Produto, área rolável onde entram os produtos e rodapé com total e Ver Resumo.
item_produto.xml	Um cartão da lista: CheckBox, nome, preço e botão Editar. É repetido para cada produto do banco.
activity_produto.xml	Tela de cadastro/edição: campos nome e preço, botões Salvar, Excluir produto e Cancelar.
activity_resumo.xml	Resumo: total em destaque, barra de progresso, mensagem colorida e cartões de orçamento e saldo.
activity_dicas.xml	Dicas de economia, integrantes do grupo e botão Voltar ao início.

Fundos e formas (`app/src/main/res/drawable/`)

Arquivo	Para que serve
fundo_gradiente.xml	Degradê verde da tela inicial.
fundo_cabecalho.xml	Cabeçalho verde com os cantos de baixo arredondados.
fundo_rodape.xml	Rodapé da lista com os cantos de cima arredondados.
fundo_cartao.xml	Cartão branco arredondado (usado em quase todas as telas).
fundo_circulo.xml	Círculo verde claro atrás dos ícones  e  .
fundo_mensagem.xml	Caixa arredondada da mensagem do resumo e das dicas.
barra_progresso.xml	Barra de progresso com pontas arredondadas (a cor é trocada pelo Java).

Valores e configuração

Arquivo	Para que serve
values/colors.xml	Todas as cores do app (verde escuro, verde dos botões, branco, vermelho de alerta...).
values/strings.xml	Todos os textos das telas, dicas, mensagens e nomes dos integrantes.
values/themes.xml	Tema do app (Material 3 claro) com a cor principal verde.
values-night/themes.xml	Tema do modo escuro – igual ao claro para manter o visual verde.
AndroidManifest.xml	Registra as 5 Activities; define a MainActivity como a tela que abre primeiro.

7. Testes automatizados

Os testes ficam em `app/src/androidTest/java/com/example/carrinhocerto/` e rodam no emulador. Os dois passaram.

Teste	O que verifica	Resultado
ProdutoCrudTest teste do banco	Usa o <code>DatabaseHelper</code> direto: insere um produto, busca e lista, atualiza nome/preço e exclui, conferindo cada etapa. No final o banco fica como estava.	✓ Passou
ProdutoTelaTest teste dos botões (Espresso)	Usa o app como uma pessoa: digita 500 e toca em Começar → + Novo Produto → preenche e Salva → confere na lista → Editar → altera e Salva → confere → Editar → Excluir produto → Sim → confere que sumiu.	✓ Passou

Como rodar: abra o arquivo do teste no Android Studio e clique no  verde ao lado do nome do teste (com o emulador ligado).

Atenção: ao terminar os testes, o Android Studio desinstala o app do emulador (e o banco junto). Depois de rodar um teste, clique em  **Run** para instalar o app de novo.

8. Resumo das regras atendidas

Regra	Status
Todas as Activities estendem AppCompatActivity e estão no AndroidManifest.xml	✓
Layouts em XML com LinearLayout dentro de ScrollView, sem bibliotecas externas	✓
Navegação entre telas com Intent	✓
Valores no padrão brasileiro (R\$ 28,90)	✓
Visual verde escuro, botões verdes com texto branco, cores no colors.xml	✓
Banco SQLite com CRUD completo (cadastrar, listar, editar e excluir com confirmação)	✓
Código comentado em português	✓